The Journal of Open Source Software

Lammps_VSCODE: Interactive editor extension for LAMMPS molecular dynamics scripting

Thomas Friedrich [1] **and Arnaud Allera** [2]

1 Ernst-Abbe-University of Applied Sciences, Jena, Germany **2** ASNR/PSN-RES/SEMIA/LSMA Centre d'études de Cadarache, F-13115, Saint Paul-lez-Durance, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Lammps_VSCODE is an extension package for the popular open-source code editor Visual Studio Code (VS Code) and other Open VSX-compatible editors (such as VS Codium). It is designed to support materials scientists working with the molecular dynamics software LAMMPS. LAMMPS is an extensive, open-source molecular dynamics package with nearly 1000 high-level commands in its scripting environment, each having numerous possible arguments. Its code base is constantly evolving, with more than 250 contributors and regular release cycles. Employing these commands in LAMMPS input scripts can be challenging, especially for beginners or users usually accessing the funtionality through intermediate packages, e.g., allowing for high-throughput simulations. Lammps_VSCODE provides an interface to LAMMPS and its extensive documentation and script syntax in an unified environment. It implements functionalities such as syntax highlighting, tooltips, embedded offline documentation for all LAMMPS commands, autocomplete suggestions, some pre-run checks, and a simulation dashboard with interactive plotting of log data as well as basic 3D visualizations of atomic structure files. The extension is updated automatically through a continuous integration pipeline on a monthly schedule to ensure ongoing compatibility with the latest official LAMMPS release. Much of the syntax and functionality is also used in derivative packages such as LIGGGHTS (Kloss et al., 2012), where **Lammps_VSCODE** can also be used accordingly.

Statement of need

LAMMPS stands for "Large-scale Atomic/Molecular Massively Parallel Simulator" and is a classical molecular dynamics code. The development began in the 1990s at Sandia National Laboratories (Plimpton, 1995) and continued from 2004 as an open source project, with a large number of contributors and an even larger number of users (Thompson et al., 2022). The scripting language used for setting up LAMMPS simulations has 1387 documented commands in the release branch of the GitHub repository at the time of writing, and several parsing rules, which add a layer of complexity. The actual scripts are usually not particularly extensive and consist of four main parts:

1. Initializations
2. System definitions
3. Simulation settings
4. Simulation execution

However, in principle scripts can have arbitrary complexity and several features and conventions may not be obvious to new users. For example, compute commands or fixes with a given ID may be referenced by their ID with prefixes, such as `c_ID` and `v_ID` respectively, without declaring these variables explicitly. Variables of different types may be defined explicitly and referenced in the script. Expressions can also be evaluated directly without declaring variables,

such as `$((xlo/xhi)/2+sqrt(v_area))`. In this example `$(...)` indicates a direct evaluation, `xlo`, `xhi` are built-in variables, `v_area` is a user-defined variable, `sqrt()` is a LAMMPS built-in function, and mathematical operators are used and work as expected. Many more such rules may be found in the documentation. When invoking a particular command, certain additional keywords can be passed to customize its effects, some keywords needing to be followed by values which can be flags, strings, numbers, etc. As arguments are interpreted implicitly from their position, the resulting scripts can be challenging to comprehend. Additionally, partly due to limitations in simulation codes interfaces, and to support high-throughput simulation, modern workflows increasingly avoid explicitly writing LAMMPS scripts –either using the LAMMPS Python interface, or packages such as pyiron (Janssen et al., 2019). The increasing popularity of large language models (LLM) assistants for developing code is another trend susceptible to reduce LAMMPS users proficiency with the scripting language, and to produce plausible-looking, yet incorrect, code.

In this context, **Lammps_VSCODE** restores human interpretability and control, and eases error-prone research workflows. Quick access to the extensive documentation is essential for productivity and learning. The LAMMPS scripting environment defines a rich domain-specific language, which syntax highlighting and tooltips can significantly help navigate. A direct interface for running and evaluating simulations within a familiar development environment supports this further. The advantages **Lammps_VSCODE** provides are appreciated by thousands of users and recommended in several online tutorials or packages (Chapman, 2024; Fahim, 2022; Mihok, 2022).

State of the field

Editor support for LAMMPS scripting has historically been limited. A few syntax highlighting packages for Sublime Text exist (BYU-BMR, 2017; Choudhary, 2017), but these have been unmaintained since 2017 and provide only basic keyword highlighting and snippets. An Emacs mode for LAMMPS, `lammps-mode` (Goswami, 2019), is available and provides syntax highlighting derived from `shell-script-mode`. One other VS Code extension for LAMMPS, `lammps-lang`, was published in 2019 but has been abandoned and offers only rudimentary syntax highlighting. None of these tools provide features beyond basic highlighting. A currently unmaintained linter for the LAMMPS language was designed as a complement to the present solution (Chapman, 2024). **Lammps_VSCODE** is the only tool that offers comprehensive language support including autocompletion, embedded offline documentation, hover information, linting, a task provider for running simulations, and an interactive simulation dashboard with 3D visualization of atomic structures and live plotting of thermodynamic output.

Software design

Lammps_VSCODE is implemented in TypeScript using the VS Code Extension API. The extension is organized into modular components: a TextMate grammar file (`lmps.tmLanguage.json`) for syntax highlighting, providers for autocompletion, hover information, and diagnostics (linting), a documentation panel that renders LAMMPS command documentation as HTML within VS Code, and a simulation dashboard built with WebView panels for interactive plotting and 3D visualization.

A key design decision is the automated synchronization with the official LAMMPS documentation. A continuous integration pipeline runs monthly, parsing the reStructuredText documentation from the LAMMPS repository into structured JSON objects that drive the autocompletion, hover, and documentation features. This ensures that the extension stays current with the rapidly evolving LAMMPS codebase without requiring manual updates. The grammar file for syntax highlighting is also regenerated during this process from the latest

command list. This approach was chosen over static keyword lists precisely because LAMMPS evolves continuously with frequent additions of new commands and styles.

Research impact statement

Since its initial release in 2019, **Lammps_VSCODE** has been installed over 47,000 times from the VS Code Marketplace. It has been recommended in several community tutorials on setting up LAMMPS workflows (Fahim, 2022; Mihok, 2022) and is used by researchers across materials science, chemistry, and physics. The extension lowers the barrier to entry for new LAMMPS users, by providing an integrated development environment with immediate access to documentation, error checking, and simulation monitoring. Its broad adoption is evidenced by active community engagement through GitHub issues and feature requests from users worldwide.

AI usage disclosure

The repository was designed and implemented over years by individuals without the use of AI tools. Some current commits, especially the creation of tests were assisted by GitHub Copilot using Anthropic's Claude Opus 4.5.

Acknowledgements

We want to acknowledge the many GitHub-users who contributed enhancement-ideas and bug-reports. We are particularly grateful for the anonymous donations to Ecologi through the Treeware license of **Lammps_VSCODE**.

References

BYU-BMR. (2017). *Lammps-st3: LAMMPS syntax highlighting for Sublime Text 3*. https://github.com/BYU-BMR/lammps-st3

Chapman, A. (2024). *Lammps-analyser: A linter for the LAMMPS scripting language*. https://github.com/chappertron/lammps-analyser

Choudhary, K. (2017). *Lammps-st3: LAMMPS syntax highlighting and snippets for Sublime Text 3*. https://github.com/ipcamit/lammps-st3

Fahim, A. (2022). Getting started with LAMMPS. In *Abrar's Space*. Blog Post. https://abrarfahim.com/getting-started-with-lammps-2

Goswami, R. (2019). *Lammps-mode: Emacs syntax highlighting for LAMMPS*. https://github.com/HaoZeke/lammps-mode

Janssen, J., Surendralal, S., Lysogorskiy, Y., Todorova, M., Hickel, T., Drautz, R., & Neugebauer, J. (2019). Pyiron: An integrated development environment for computational materials science. *Computational Materials Science*, *163*, 24–36. https://doi.org/https://doi.org/10.1016/j.commatsci.2018.07.043

Kloss, C., Goniva, C., Hager, A., Amberger, S., & Pirker, S. (2012). Models, algorithms and validation for opensource DEM and CFD–DEM. *Progress in Computational Fluid Dynamics, an International Journal*, *12*(2-3), 140–152. https://doi.org/10.1504/PCFD.2012.047457

Mihok, F. (2022). LAMMPS workflow. In *HackMD*. Blog Post. https://hackmd.io/@Fercik/LAMMPS/_workflow

DRAFT

Plimpton, S. (1995). Fast parallel algorithms for short-range molecular dynamics. *Journal of Computational Physics*, *117*(1), 1–19. https://doi.org/10.1006/jcph.1995.1039

Thompson, A. P., Aktulga, H. M., Berger, R., Bolintineanu, D. S., Brown, W. M., Crozier, P. S., Veld, P. J. in 't, Kohlmeyer, A., Moore, S. G., Nguyen, T. D., Shan, R., Stevens, M. J., Tranchida, J., Trott, C., & Plimpton, S. J. (2022). LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Comp. Phys. Comm.*, *271*, 108171. https://doi.org/10.1016/j.cpc.2021.108171

DRAFT